

深度学习

张烨菲

2025年12月

第九章 深度生成对抗网络

- **9.1 生成对抗网络**
- **9.2 小波生成对抗网络**
- **9.3 多尺度生成对抗网络**
- **9.4 GAN实例**

9.1 生成对抗网络

模型目标:

- 训练数据集的模型: $\boldsymbol{x} \sim p_{\text{train}}(\boldsymbol{x})$
- 生成样本的模型: $\boldsymbol{x} \sim p_{\text{model}}(\boldsymbol{x})$
- 令 $p_{\text{model}}(\boldsymbol{x}) = p_{\text{data}}(\boldsymbol{x})$

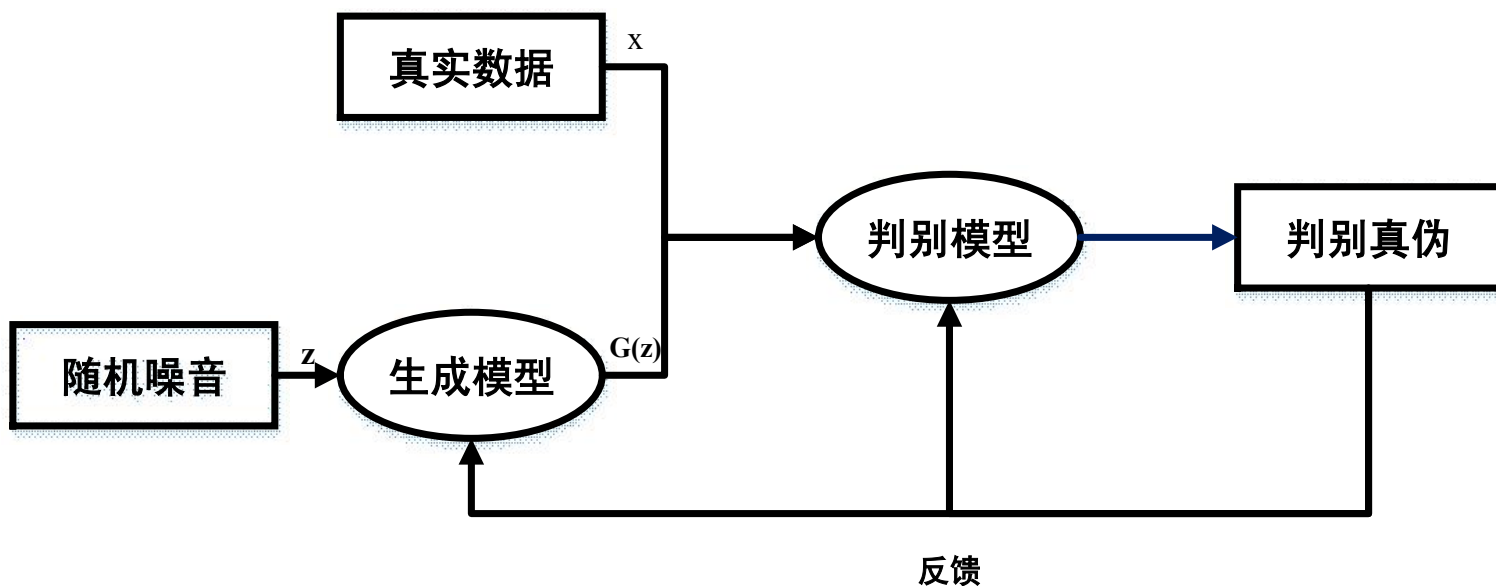
解决问题:

- 构建高维、复杂概率分布
- 数据缺失
- 多模态输出
- 真实输出任务
- 未来数据预测

9.1 生成对抗网络

9.1.1 GAN原理

生成式对抗网络（Generative Adversarial Network）是将判别模型与生成模型有机地进行融合的深度神经网络框架，其包含的生成器（估计数据分布）与判别器（判断数据真伪）通过相互竞争或对抗的方式提取数据蕴含的内在规律。



9.1 生成对抗网络

9.1.1 GAN原理

生成式对抗网络的基本原理为：首先根据随机噪声利用生成器生成新图像，然后利用判别器判别一幅图像的真实性（如图像的判别概率为1则为真、为0则为假）；在训练过程中，生成器尽量生成真实图像以欺骗判别器，而判别器则尽量把生成器生成的图像与真实图像分开，两者进而构成了一个动态的“博弈过程”。最终，生成器可生成“以假乱真”的图像（判别器难以进行判别或判别概率为1）。

9.1 生成对抗网络

9.1.1 GAN原理

GAN是怎么通过对抗的方式优化各自的模型？

下面以故事的形式解释GAN是什么，它的对抗体现在哪里：有这样一个城市，那里有一个小偷培训机构和警察局。

第一天，小偷培训机构随便训练了一下员工，就派出去执行赚钱任务，然而出去的全部小偷很快就被警察从人群中认出来并抓入监狱。

第二天，小偷机构在损失大量人手的情况下，痛改前非，认真研究训练制度，培训了一批更加专业的小偷，将其放入市场，终于有一些小偷躲过了警察的识别，并带回了部分战利品。

第三天，警察机构在收到投诉之后，立刻进行警员培训，而小偷机构在尝到甜头后，继续派出手下，然而，这次的手下在强化了职业技能的警察手里损失惨重。

第四天，小偷机构在损失大量人手的情况下，痛改前非.....

第五天，警察机构在收到投诉之后，立刻进行警员培训.....

.....

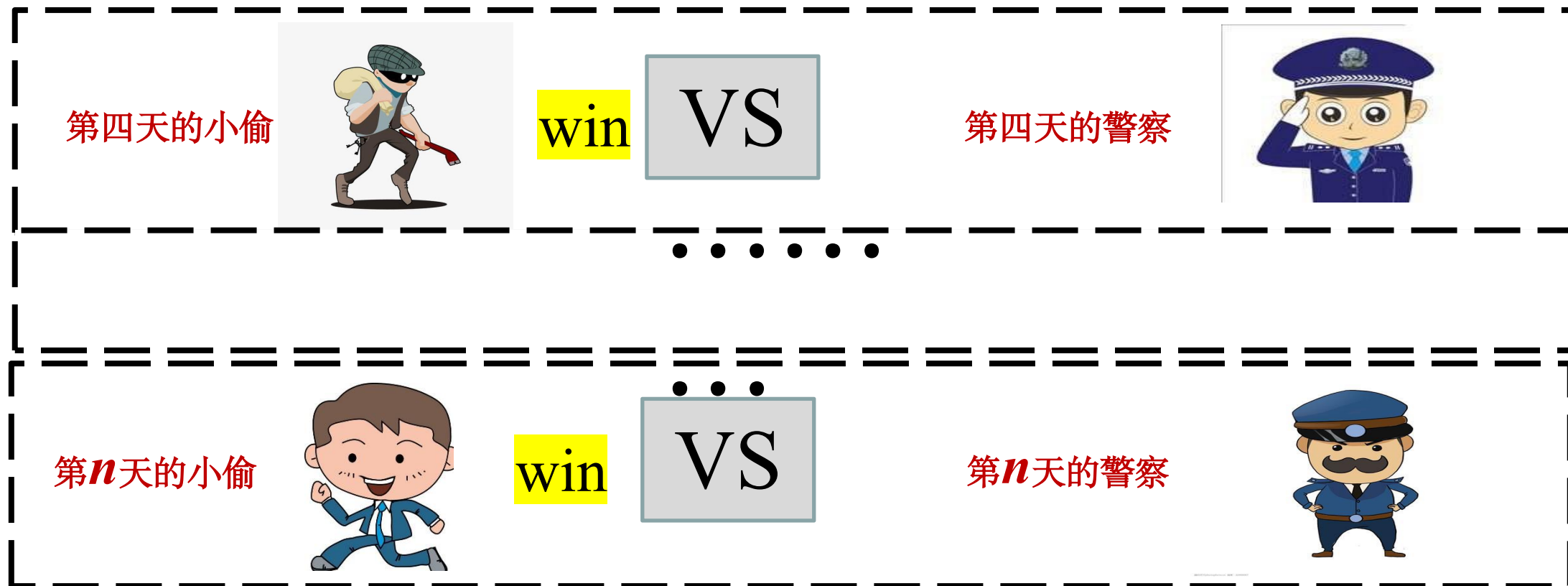
9.1 生成对抗网络

9.1.1 GAN原理



9.1 生成对抗网络

9.1.1 GAN原理

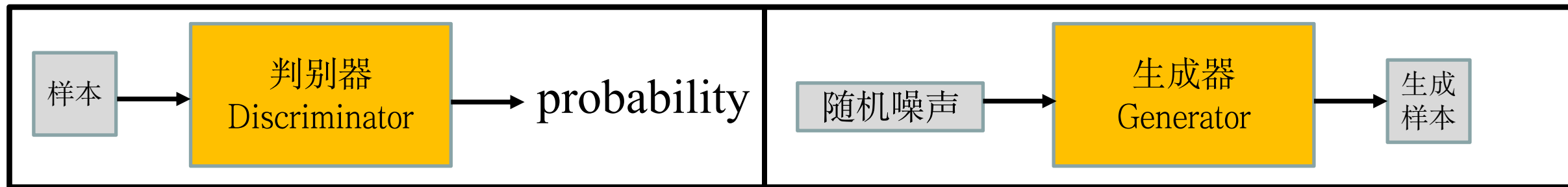


最终培训的小偷已经变成与常人无异，警察组织无法抓捕到任何小偷了，这种对抗的思想使得获得最强的小偷培训组织。

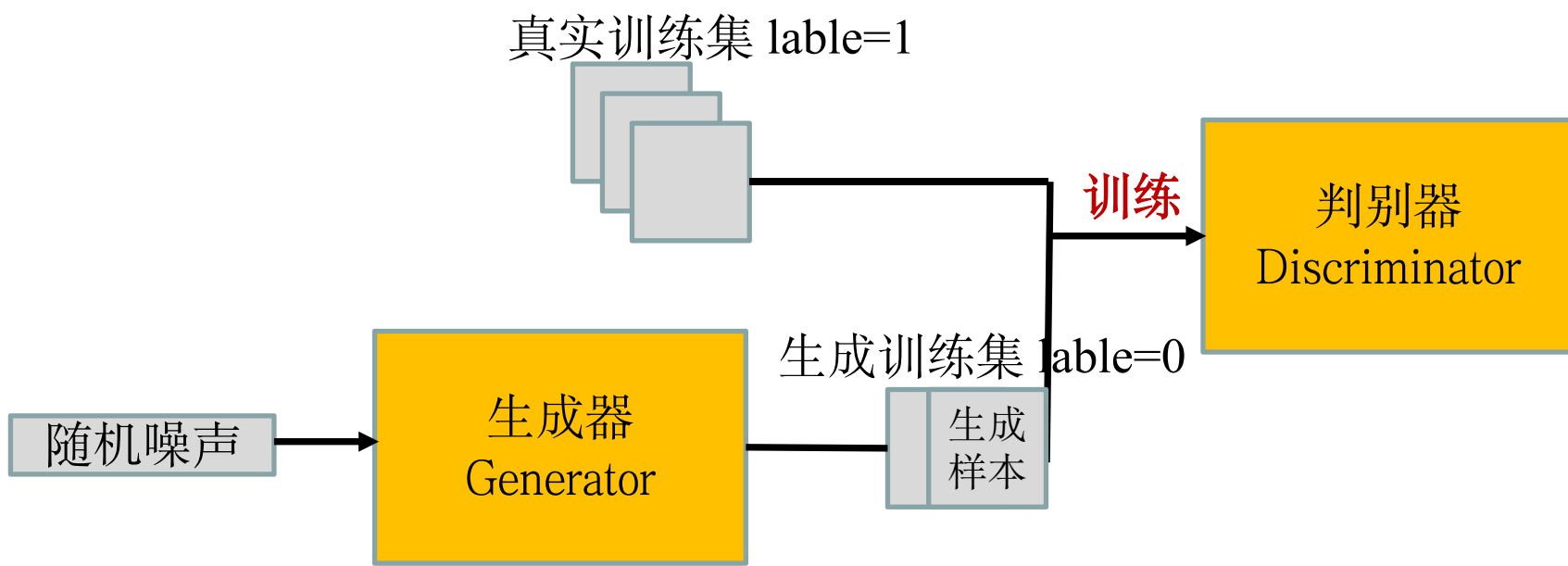
9.1 生成对抗网络

9.1.1 GAN原理

判别器训练



1. 训练判别器D (警察)



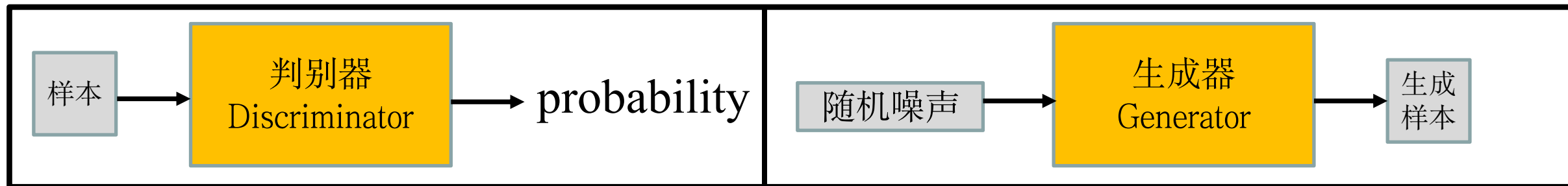
判别器D的训练集由两部分组成

- ①人为的贴标签为1的真实训练集
- ②人为的贴标签为0的生成器生成的假样本

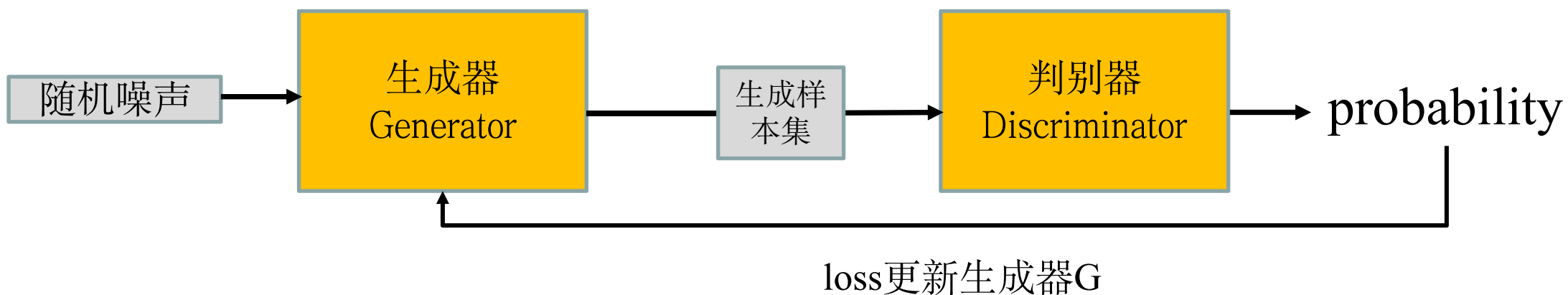
9.1 生成对抗网络

9.1.1 GAN原理

生成器训练



2. 训练生成器G (小偷培训方法)



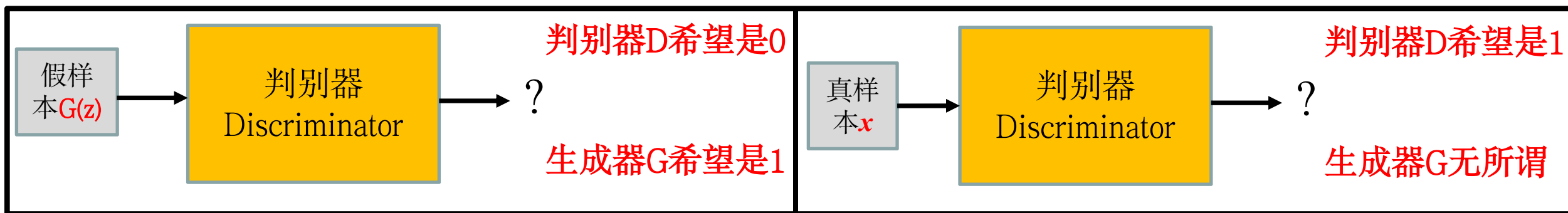
9.1 生成对抗网络

9.1.1 GAN原理

GAN的优化函数

我们已经基本知道GAN中生成器G和判别器D各自的输入及其输出，

我们假设判别器D对给定样本的输出概率值越接近1表示判别器D越坚信其属于真样本，越接近0则表示判别器D越坚信其属于假样本。



生成器G希望其生成的假样本输入给判别器D时,判别器能给出接近1的值;

判别器D希望能够正确的分别假样本和真样本。

9.1 生成对抗网络

9.1.1 GAN原理

GAN的优化函数

- GAN目标函数定义为:

$$\min_G \max_D \text{Loss}(D, G) = \mathbb{E}_{x \sim f_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim f_z(z)} [\log(1 - D(G(z)))]$$

- 最大化的目标函数为:

$$\begin{aligned} \text{Loss}(D, G) &= \int_x f_{\text{data}}(x) \log(D(x)) dx + \int_z f_z(z) \log(1 - D(g(z))) dz \\ &= \int_x f_{\text{data}}(x) \log(D(x)) + f_g(x) \log(1 - D(x)) dx \end{aligned}$$

- 给定任意生成器G, 判别器D的训练准则都是使Loss(D,G) 最大化。
- 最优判别器的计算公式为:

$$D_{G\text{-opt}}(x) = \frac{f_{\text{data}}(x)}{f_{\text{data}}(x) + f_g(x)}$$

9.1 生成对抗网络

9.1.1 GAN原理

将最优判别器代入目标函数 $\text{Loss}(\mathbf{D}, \mathbf{G})$ 中，消去 $\mathbf{D}$ ，获得 $\mathbf{G}$ 的目标函数为

$$\begin{aligned}\text{Loss}(\mathbf{G}) &= \max_{\mathbf{D}} \text{Loss}(\mathbf{G}, \mathbf{D}) \\ &= \mathbb{E}_{x \sim f_{\text{data}}} [\log D_{G-\text{opt}}(x)] + \mathbb{E}_{z \sim f_z} [\log(1 - D_{G-\text{opt}}(G(z)))] \\ &= \mathbb{E}_{x \sim f_{\text{data}}} [\log D_{G-\text{opt}}(x)] + \mathbb{E}_{x \sim f_g} (\log(1 - D_{G-\text{opt}}(x))) \\ &= \mathbb{E}_{x \sim f_{\text{data}}} \left[\log \frac{f_{\text{data}}(x)}{f_{\text{data}}(x) + f_g(x)} \right] + \mathbb{E}_{x \sim p_g} \left[\log \frac{f_g(x)}{f_{\text{data}}(x) + f_g(x)} \right]\end{aligned}$$

当 $f_g = f_{\text{data}}$ ， $\mathbf{D}=0.5$ 时判别器无法区分真实样本和生成样本，此时目标函数 $\text{Loss}(\mathbf{G})=-\log(4)$ 。将目标函数转换为KL散度的形式，得

$$\text{Loss}(\mathbf{G}) = -\log(4) + KL\left(f_{\text{data}} \parallel \frac{f_{\text{data}} + f_g}{2}\right) + KL\left(f_g \parallel \frac{f_{\text{data}} + f_g}{2}\right)$$

将目标函数中的两个KL散度转换为JS散度，得

$$\text{Loss}(\mathbf{G}) = -\log(4) + 2 \cdot \text{JSD}(f_{\text{data}} \parallel f_g)$$

9.1 生成对抗网络

9.1.2 实现框架

步骤1: G接收一个随机的噪声 z 并生成数据，D判断生成的数据是不是“真实的”。

步骤2: 计算生成模型G的损失函数。其中， $G(z)$ 表示生成模型的输出， $D(G(z))$ 表示判别模型D判断生成模型G生成的数据是否为真实的概率。

步骤3: 计算判别模型D的损失函数。其中， x 表示输入参数，即真实样本数据， $D(x)$ 表示判别模型的输出，即输入 x 为真实数据的概率。

步骤4: 计算优化函数

$$\min_G \max_D \text{Loss}(D, G) = \mathbb{E}_{X \sim f_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{Z \sim f_z(z)} [\log(1 - D(G(z)))]$$

9.1 生成对抗网络

9.1.2 实现框架

步骤5: 在训练过程当中，G尽可能生成真实的数据去欺骗D，而D则尽量把G生成的数据和真实的数据区分开来，最终生成模型和判别模型形成了一个动态的“博弈过程”。

步骤6: 判断D是否能判别G所生成的数据为真实，若能，则得到训练好的数据，否则，需要调整输入参数 x 后继续执行步骤2。

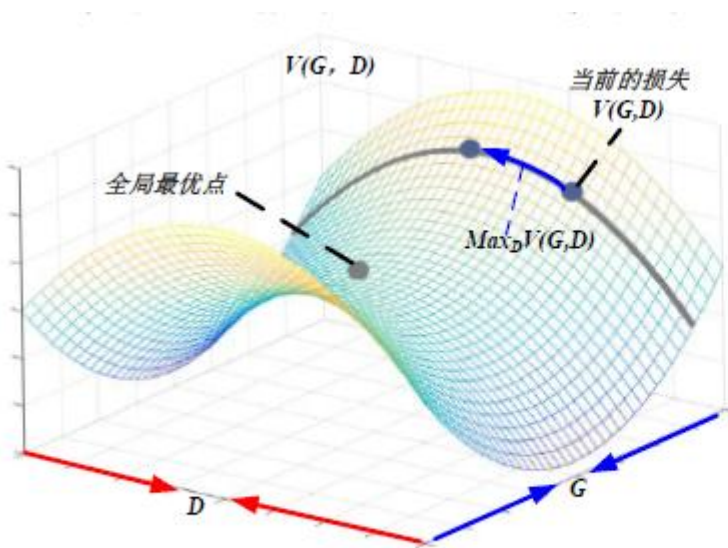
步骤7: 理想状态下，G生成足以“以假乱真”的 $G(z)$ ，对于D而言，无法判定G所生成的数据是否真实，即 $D(G(z))=0.5$ 。

9.1 生成对抗网络

9.1.2 实现框架

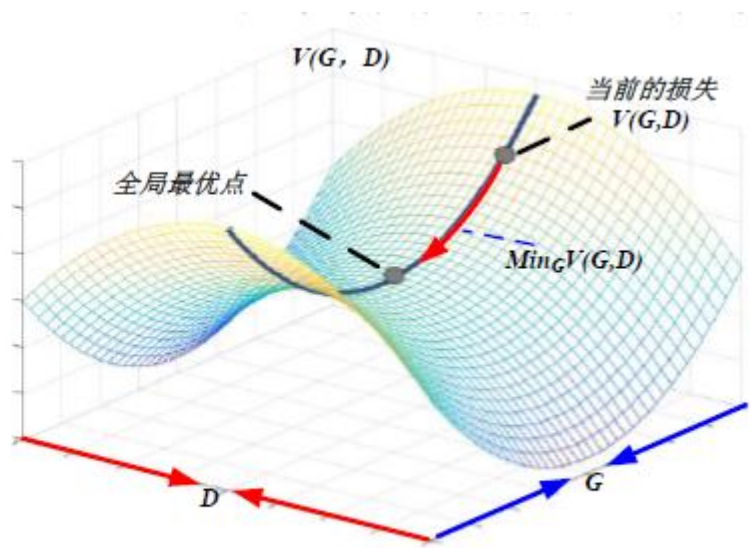
GAN的优化函数

GAN的判别器和生成器是交替优化的
当更新判别器的参数时，需要固定生成器；
当更新生成器的参数时，需要固定判别器。如下图所示：



(a) D的优化过程

最大化 $V(D, G)$



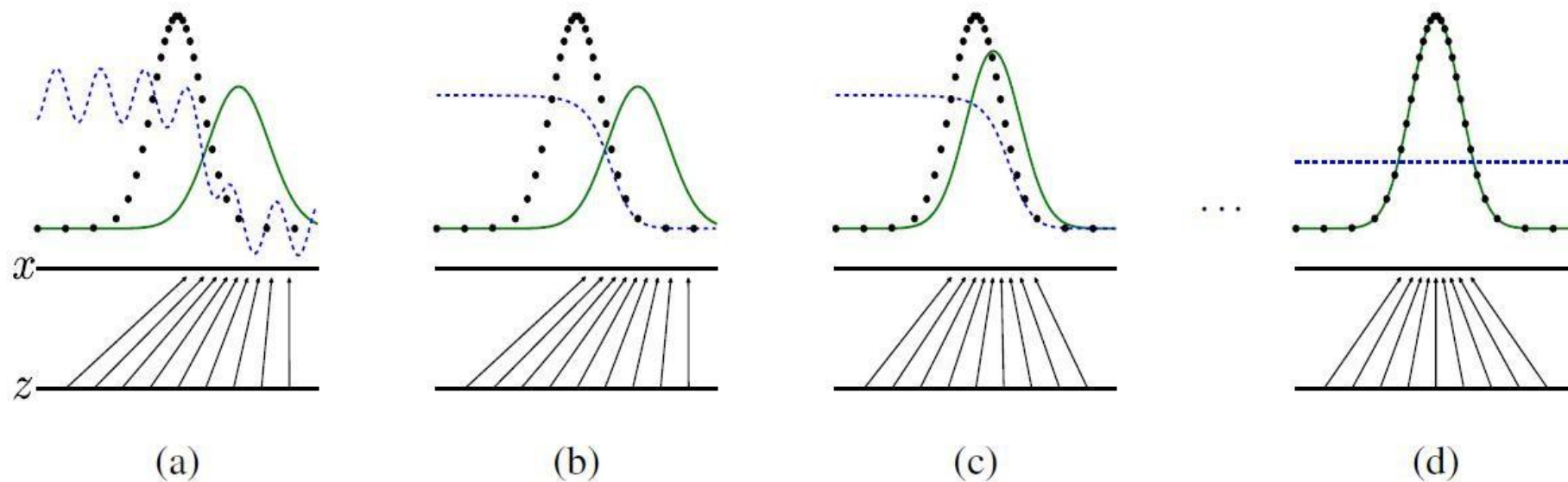
(b) G的优化过程

最小化 $V(D, G)$

9.1 生成对抗网络

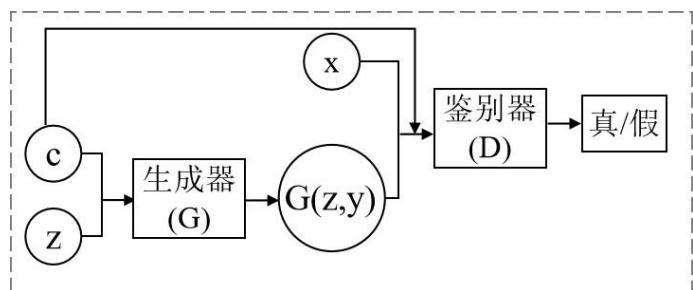
9.1.2 实现框架

训练过程

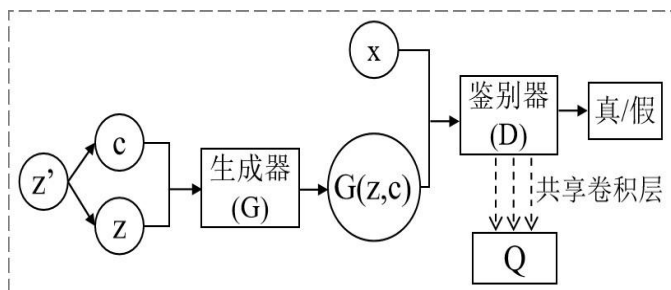


其中：绿线表示生成器的数据分布；黑线表示真实数据分布；蓝线表示判别器的结果分布

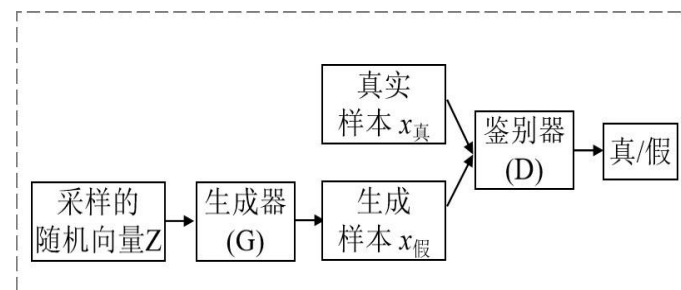
9.1 生成对抗网络



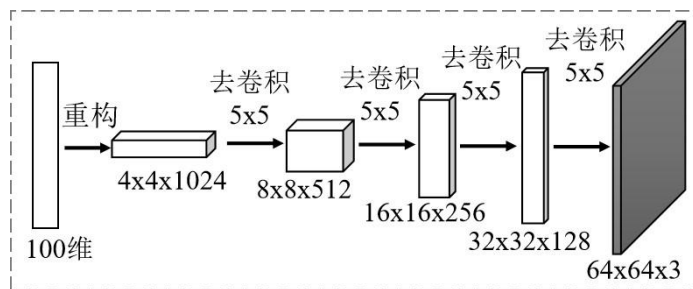
CGAN结构



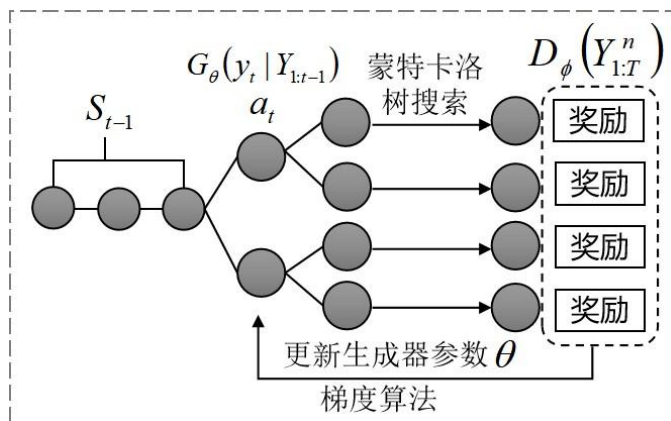
InfoGAN结构



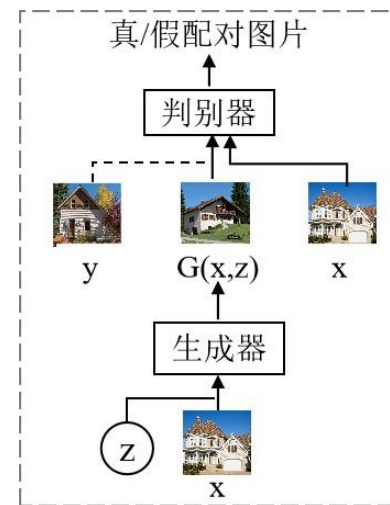
WGAN结构



DCGAN结构



SeqGAN结构



Pix2Pix结构

9.2 小波生成对抗网络

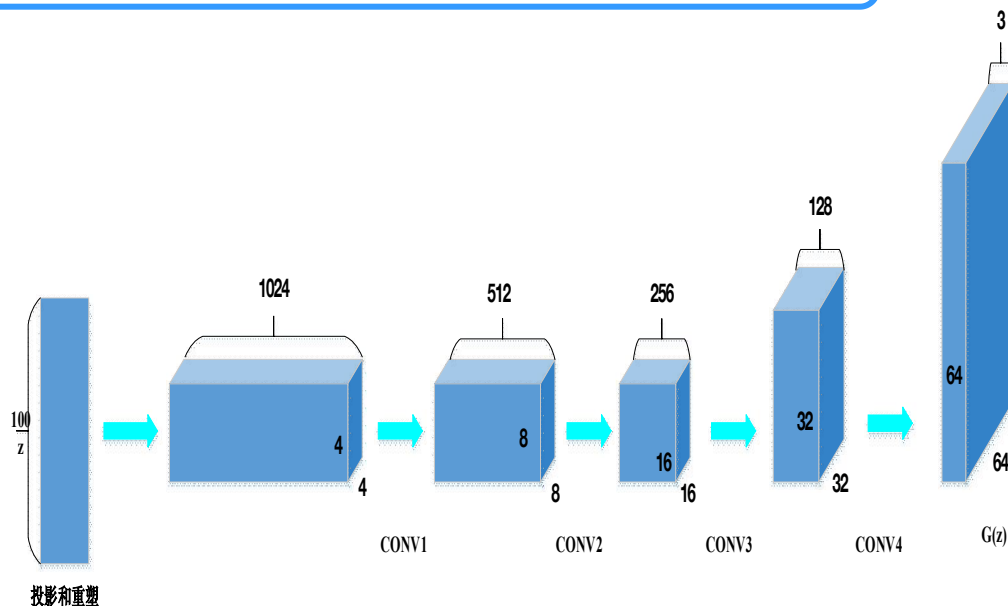
9.2.1 深度卷积生成对抗网络

DCGAN四点架构设计规则

- (1) 使用卷积层替代池化层。
- (2) 去除全连接层。
- (3) 使用批归一化(Batch Normalization)。
- (4) 使用恰当的激活函数。生成器中使用ReLU函数，判别器中使用LReLU函数。

DCGAN中生成器G的网络结构

生成器G的输入是1个100维的随机数据 z ，服从在 $[-1, 1]$ 的均匀分布。生成器G网络的第一层为全连接层，其任务是将100维的噪声向量变成 $4 \times 4 \times 1024$ 维的向量，并从第二层开始使用步长卷积做上采样操作，逐步减少通道数，最终的输出为 $64 \times 64 \times 3$ 的图像。



9.2 小波生成对抗网络

9.2.2 小波深度卷积生成对抗网络

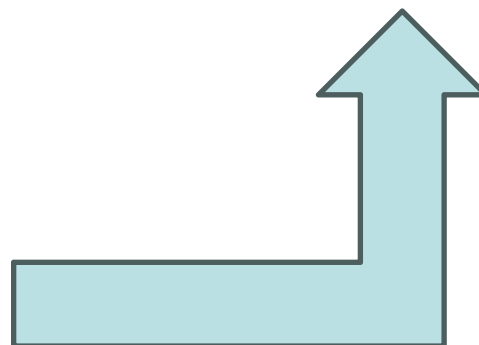
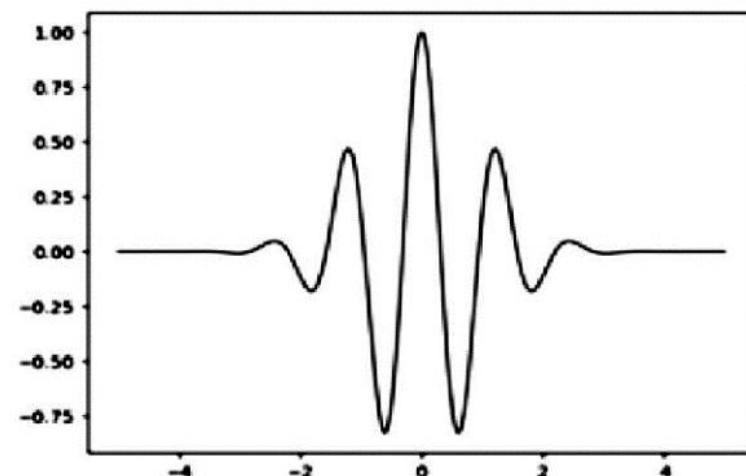
1. 小波函数构造

小波变换具有多分辨分析的特点，在时频两域都具有表征信号局部特征的能力，在DCGAN网络的生成模型中，为了获得原始图像的更多局部特征，用非线性小波基取代了DCGAN生成模型第一个卷积层激活函数，在小波函数的选取上考虑传统激活函数有相似性质的小波基。

小波中母小波种类很多，其中Morlet小波振幅光滑且连续，如果选择Morlet小波作为激活函数来构建小波深度卷积生成对抗网络，其函数表达式为

$$\psi(t) = Ce^{-\frac{t^2}{2}} \cos(5t)$$

式中，C是重构时的归一化常数。Morlet小波没有尺度函数，而且是非正交分解，时域波形如右图



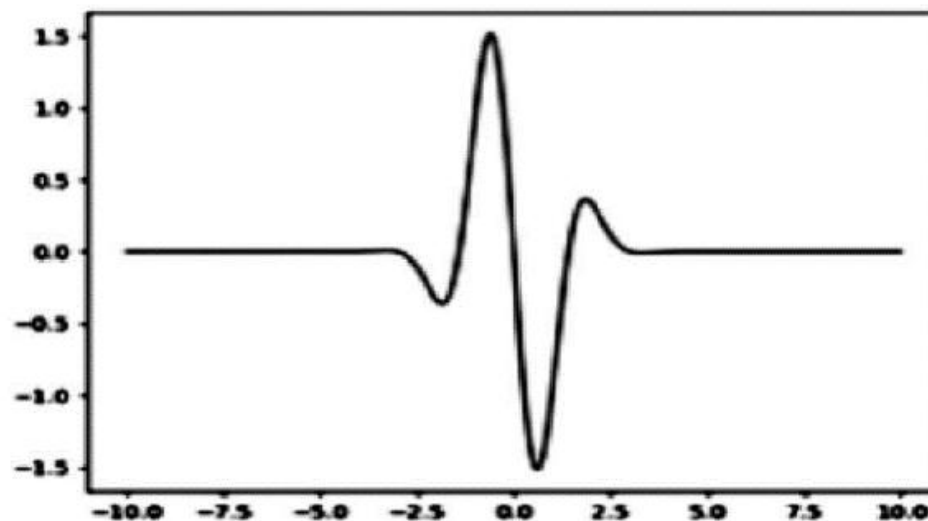
9.2 小波生成对抗网络

9.2.2 小波深度卷积生成对抗网络

1. 小波函数构造

$$\psi(t) = -\frac{7}{4}\sin\left(\frac{7}{4}t\right)e^{-\frac{t^2}{2}} - t\cos\left(\frac{7}{4}t\right)e^{-\frac{t^2}{2}}$$

其相应时域波形

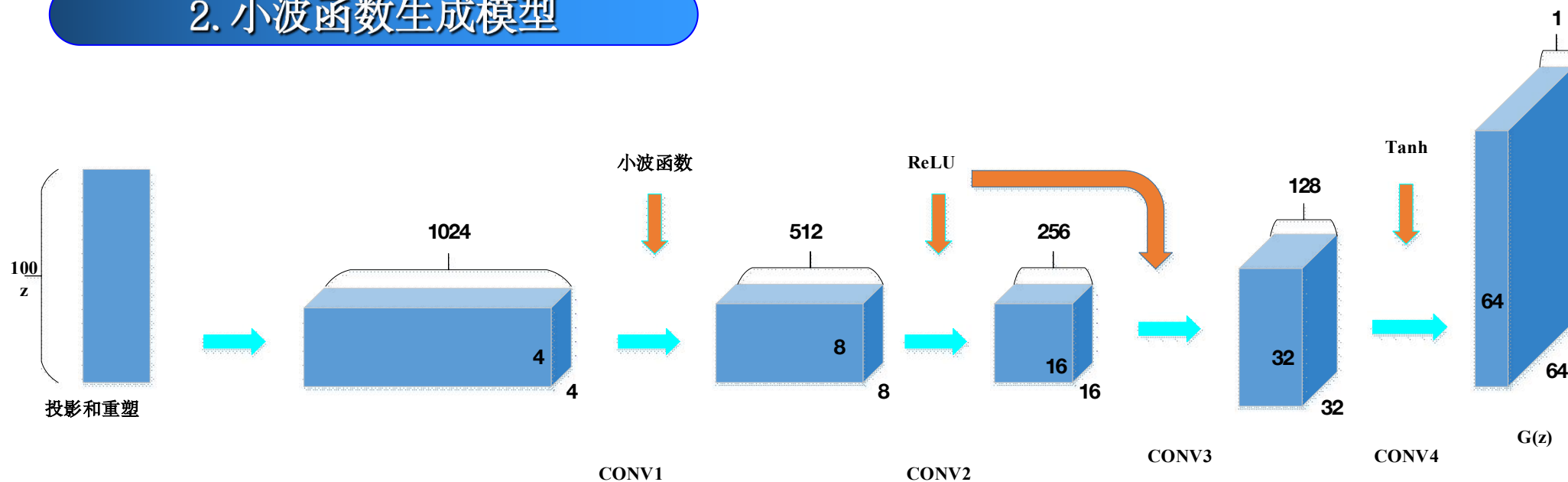


修改的小波时域图

9.2 小波生成对抗网络

9.2.2 小波深度卷积生成对抗网络

2. 小波函数生成模型



加入小波函数的生成器结构

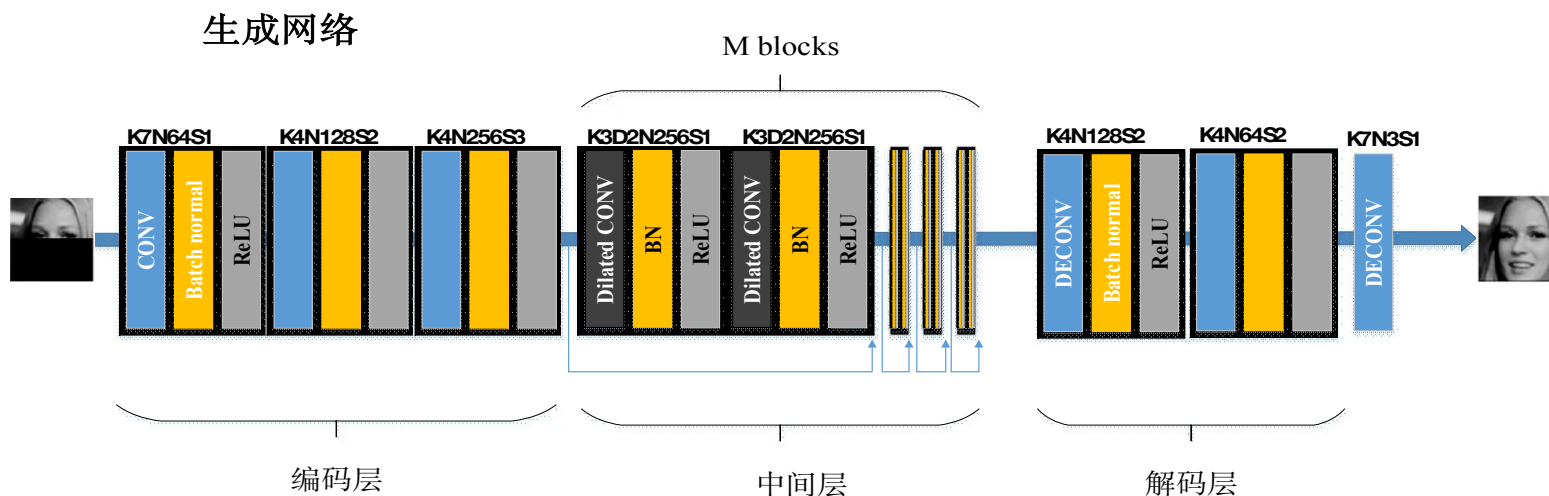
将修改的小波函数加入到生成器中的第一个卷积层，代替激活函数。

9.2 小波生成对抗网络

9.2.3 高频小波生成对抗网络网络

1. 图像补全GAN网络

(1) 图像补全生成器网络结构。

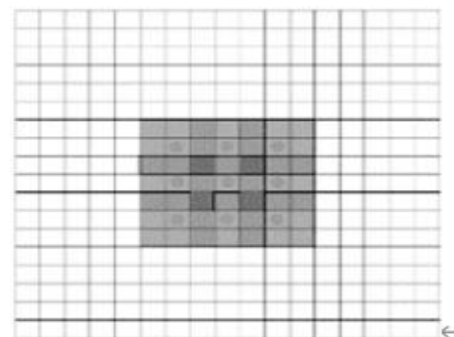
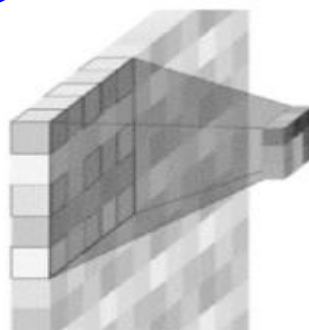
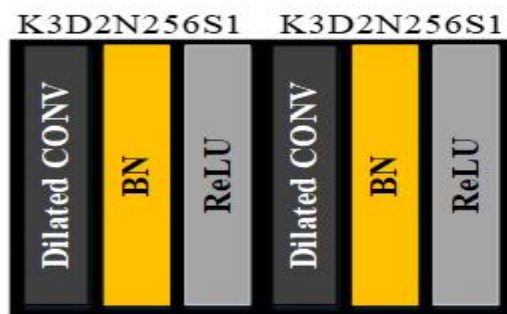


在图像识别任务中，遮挡物覆盖了真实图像区域，导致识别的信息有缺损，网络无法做出准确判断。而图像补全可以在一定程度上弥补这种信息缺损，让网络对完整的图像信息进行识别。如果考虑的遮挡区域相对固定，则在训练网络时，可预先定义遮挡物的位置，而不考虑对遮挡物的识别与定位。图像补全GAN网络由生成网络和判别网络构成。其中，生成网络的目标是将有遮挡的图像补全成完整的图像，如上图所示。

9.2 小波生成对抗网络

9.2.3 高频小波生成对抗网络网络

1. 图像补全GAN网络



(a)类 ResNet 模块结构 · (b)空洞率为 2 的空洞卷积 · (c)空洞率为 2 的空洞卷积的感受野

类 ResNet 模块

图中，K表示网络卷积核大小，N表示通道数目，S表示卷积核的步长，D表示空洞率。编码层的设计是为了缩小网络输入图像的空间尺寸，保证后续的网络运算能够在较小的空间尺寸下进行，提升网络的运行效率。因为空间尺寸的缩小伴随着一定的量化误差，误差越小生成的图像质量越好，所以编码层先采用K7N64S1的结构设计，即卷积核大小为 $7 \times 7 \times 64$ ，步长为1的卷积层在不缩小网络空间尺寸大小时提取图像全局特征。之后再使用两个卷积核空间大小为 4×4 ，步长为2的卷积层将网络尺寸缩小4倍同时模块采用空洞率为2的空洞卷积代替常规卷积，增加了网络的感受野，增强网络感受全局特征的能力。

9.2 小波生成对抗网络

9.2.3 高频小波生成对抗网络网络

(2) 判决器设计。

判别网络



判别器是一个二分类网络，如上图所示。输入是原始完整的图像或者生成器生成的图像；输出为一个二分类结果，判断是原始图像还是生成图像。

9.2 小波生成对抗网络

9.2.3 高频小波生成对抗网络网络

2. 损失函数

(1) 总损失设计

设 I_{gt} 为真实样本图像, M 为掩膜, M 使用掩膜 掩盖真实样本图像得到部分区域缺失的图像 $\tilde{I}$, 即

$$\tilde{I} = I_{\text{gt}} \odot (1 - M)$$

式中, $\odot$ 表示矩阵元素逐点对应相乘。将 $\tilde{I}$ 作为生成网络输入, 得到一个填充缺失区域后的彩色图像 I_p , 其分辨率与输入图像相同, 即

$$I_p = G(\tilde{I})$$

网络训练的总损失函数L定义为

$$Loss = \alpha Loss_{l_1} + \beta Loss_{\text{adv}} + \gamma Loss_{\text{pere}} + \mu Loss_{\text{style}} + \delta Loss_{\text{wavelet}}$$

式中, α 、 β 、 γ 、 μ 、 δ 为损失系数; $Loss_{l_1}$ 表示重构的 l_1 损失。

9.2 小波生成对抗网络

9.2.3 高频小波生成对抗网络网络

2. 损失函数

除重构的 l_1 损失外，在网络训练的过程中对抗损失定义保证了生成对抗的过程，使补全后的图片更加接近原图，即

$$Loss_{adv} = E_{I_{gt} \sim f(I_{gt})} [\log D(I_{gt})] + E_{I_{comp} \sim f(I_{comp})} [\log(1 - D(\tilde{I}_p))]$$

式中, $f(I_{gt})$ 表示真实样本的分布; $D(\bullet)$ 表示判决器的输出; $f(I_{comp})$ 表示生成补全图像后的生成样本分布。 $Loss_{pere}$ 为感知损失，它是以预训练网络中特征图之间的距离度量作为在感知上生成图 I_{gt} 与真实图 I_p 上不同的惩罚，定义为

$$Loss_{pere} = \frac{1}{H_i W_i C_i} \|\phi^{(i)}(I_{gt}) - \phi^{(i)}(\tilde{I}_p)\|_1$$

式中, $\phi^{(i)}$ 为预训练网络中第 i 层网络输出的特征图，如使用在ImageNet上预训练的VGG19网络的 ReLU层输出的特征图。同时这些特征图还用于计算风格损失，对于给定的特征图尺寸 $H_i \times W_i \times C_i$ ，风格损失使用Gram矩阵计算：

$$Loss_{style} = \|(\phi^{(i)}(I_{gt}))^T \phi^{(i)}(I_{gt}) - \phi^{(i)}(\tilde{I}_p)^T \phi^{(i)}(\tilde{I}_p)\|_1$$

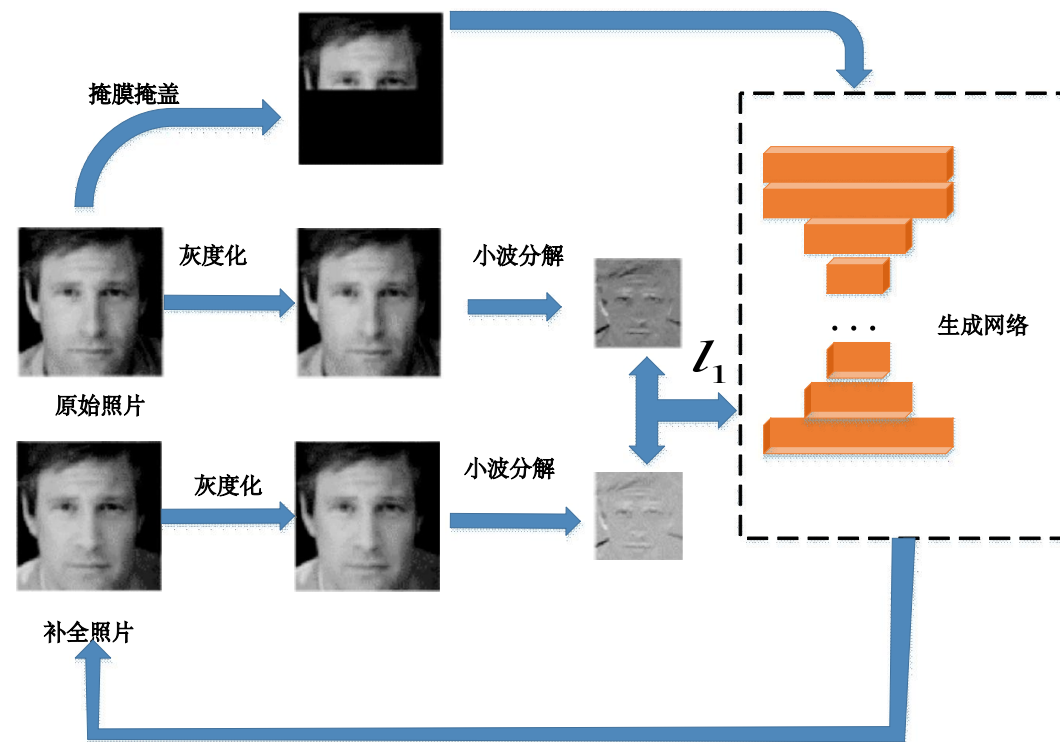
9.2 小波生成对抗网络

9.2.3 高频小波生成对抗网络网络

2. 损失函数

(2) 小波损失

像素方式的损失有 l_1 损失和均方误差 (MSE) 损失，然而在实现较高的峰值信噪比 (PSNR) 的同时，往往缺乏图像的高频信息，从而导致生成图像的纹理过度平滑，生成出的图像在视觉上不满意。针对这一问题，利用基于小波分解定义一个高频小波损失，如右图所示。



9.2 小波生成对抗网络

9.2.3 高频小波生成对抗网络网络

3. 网络训练架构

两个网络的训练架构如下：

输入：初始化判别器和生成器网络参数 θ_d 、 θ_g ，全图像（无遮挡）数据data。

输出：判别器D和生成器G。

步骤1：从全图像数据data中采样 $I_{gt1}, I_{gt2}, \dots, I_{gtm}$ 作为一个批量；遮挡下半图像，得到 $\tilde{I}_1, \tilde{I}_2, \dots, \tilde{I}_m$ 经过生成器G得到补全后的全图像数据 $\tilde{I}_{p1}, \tilde{I}_{p2}, \dots, \tilde{I}_{pm}$ 其中 $\tilde{I}_{p1}, \tilde{I}_{p2}, \dots, \tilde{I}_{pm}$ 将全图像数据和补全后的全图像数据通过判别器更新判别器D的网络参数。

步骤2：从全图像数据data中采样 $\tilde{I}_{pi} = G(\tilde{I}_i)$ 作为一个批量；遮挡下半图像，得到 $\tilde{I}_1, \tilde{I}_2, \dots, \tilde{I}_m$ ，经过生成器G得到补全后的全图数据 $I_{gt1}, I_{gt2}, \dots, I_{gtm}$ 再经过判别器D，更新生成器G的网络参数 θ_g 。

9.2 小波生成对抗网络

9.2.3 高频小波生成对抗网络网络

4. 评价指标

图像补全的通用指标衡量模型生成图像的质量，包括结构相似性指数(SSIM)、峰值信噪比(PSNR)和平均绝对误差(MAE)。假设像素间相互独立性，结构相似性指数定义为

$$SSIM = \frac{(2m_{I_{gt}} m_{I_p} + c_1)(2C_{I_{gt} I_p} + c_2)}{(m_{I_{gt}}^2 + m_{I_p}^2 + c_1)(\sigma_{I_{gt}}^2 + \sigma_{I_p}^2 + c_2)}$$

C、W、H分别为图像的通道数、宽和高，峰值信噪比由均方误差(MSE)进行定义为

$$PSNR = 20 \cdot \log_{10} \left(\frac{255}{MSE} \right) \quad MSE = \frac{1}{CWH} \| I_{gt} - \tilde{I}_p \|^2$$

9.2 小波生成对抗网络

9.2.3 高频小波生成对抗网络网络

4. 评价指标

同时，真实图像和补全图像的平均绝对误差定义为

$$MAE = \frac{1}{CWH} \|I_{\text{gt}} - \tilde{I}_{\text{p}}\|_1$$

准确率和召回率分别定义为

$$Accuracy = \frac{\sum TP + \sum TN}{AllSamples}$$

$$Recall = \frac{TP}{TP + FN}$$

式中，TP表示正确判断为真；FP表示错误的判断为真；TN表示正确判断为假；FN表示错误的判断为假；AllSamples表示样本数量。

9.3 多尺度生成多抗网络

9.3.1 湍流退化图像

恢复退化图像工作原理

在训练过程中，将湍流退化图像作为生成器G的输入，并将生成图像与训练数据中的清晰图像共同作为判别器D的输入。判别器与生成器采取单独交替训练的训练方式，通过误差回传更新网络参数，直至设定的迭代次数。

湍流退化过程的数学模型

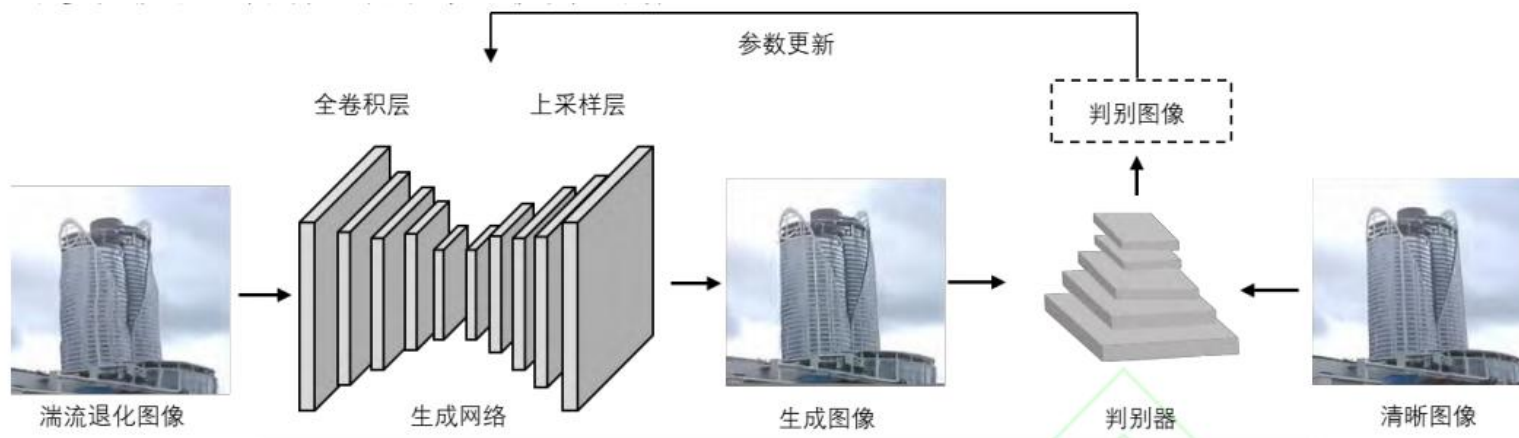
$$f(u) = D_u(H(I(u))) + n_u$$

式中， $I(u)$ 为需要复原的清晰图像， $f(u)$ 为成像设备获取的湍流退化图像； $u=(x,y)^T$ 为图像中像素的空间位置； H 为传感器光学模糊算子； D_u 为湍流畸变算子，包含局部形变和空间模糊； n_u 为加性噪声。由于湍流畸变算子同时包含模糊和形变两种模糊核，使用卷积神经网络提取像素特征时，需要设计足够大的感受野来覆盖像素区域，对提取的特征进行动态权重调节有助于模型关注重要信息。

9.3 多尺度生成对抗网络

9.3.2 多尺度GAN结构

1. 多尺度GAN结构



针对湍流图像复原任务需要同时去除几何畸变和模糊，采用多尺度生成对抗网络模型，如上图所示。生成器是一个对称的U-Net网络结构，判别器采用PatchGAN结构，由4个卷积核大小为 4×4 的卷积层构成。多尺度网络模型表现为：

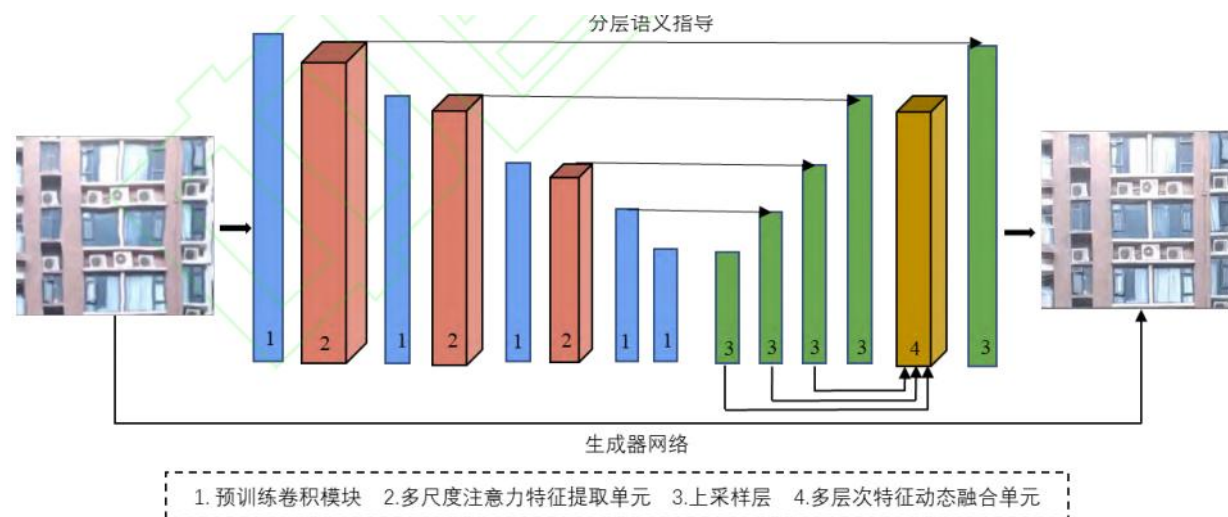
(1) 在图像特征提取上，多尺度注意力特征提取单元使用不同大小的卷积核在更大的感受野范围内提取多尺度特征信息；

(2) 在模型结构上，多层次特征融合单元对不同比例的特征图进行权重调节的动态融合，挖掘不同级别的语义信息。

9.3 多尺度生成对抗网络

9.3.2 多尺度GAN结构

生成器网络结构

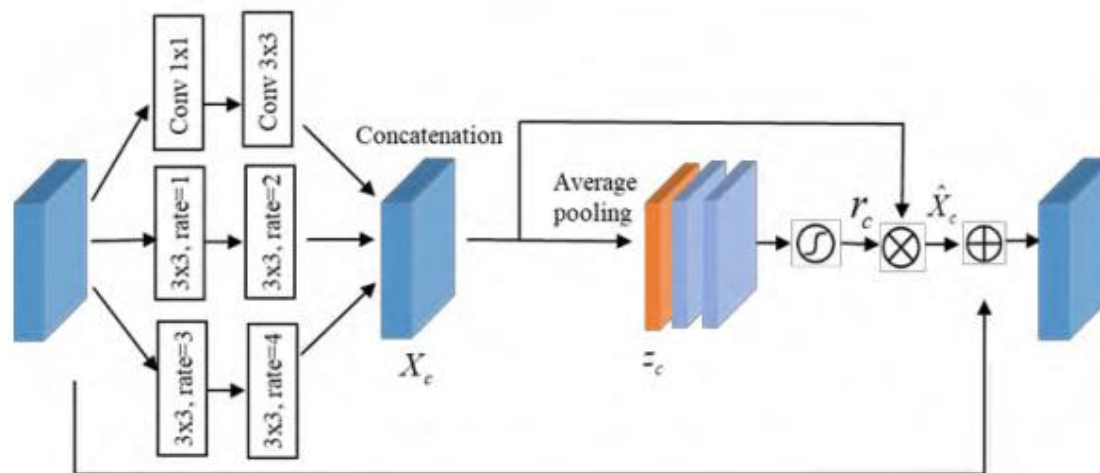


U-Net网络全卷积部分由预训练卷积模块和多尺度注意力特征提取单元组成，预训练卷积模块使用Inception-ResNet-v2网络中的卷积层与最大池化层，多尺度注意力特征提取单元提供多尺度特征信息并使用特征注意来挖掘通道的相关性。上采样部分由卷积层与上采样层组成，插入多比例特征动态融合单元将不同比例的特征图上采样到相同的输入大小，并动态调节权重连接成一个张量，加强不同尺度特征图的信息共享。在U-Net网络执行编码解码过程中会损失图像的细节特征，增加跳跃连接作为分层语义指导，将具有更多局部信息的浅层网络与对应的深层网络结合起来，更加充分的利用高层特征的语义信息和底层特征的细粒度特征，提升重建图像的视觉细节特征。生成器引入了一个直接从输入到输出跳跃连接，促使模型重点学习残差。

9.3 多尺度生成多抗网络

9.3.2 多尺度GAN结构

2.多尺度注意力特征提取单元



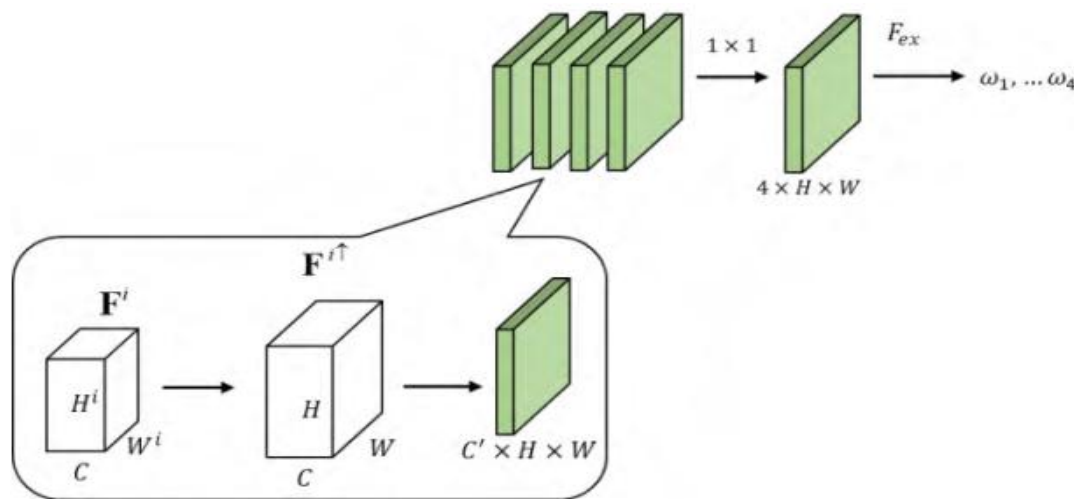
如右上图所示的多尺度注意力特征提取单元由多分支卷积层和注意力层连接而成。多分支卷积层对应不同尺寸的感受野，能够提取到多种特征，注意力层充分学习到退化图像中的重要信息，保证重建图像的准确清晰。

多分支卷积层由不同尺寸的空洞卷积并列组成，三条支路感受野分别为 3×3 、 7×7 和 15×15 ，同时对输入特征图进行特征提取，获得不同尺度的信息特征图后，通过卷积操作将级联的特征图重新调整为输入尺寸。

9.3 多尺度生成多抗网络

9.3.2 多尺度GAN结构

3. 多层次特征动态融合单元



在生成器网络的上采样部分，不同层次的特征图蕴含着不同的实例信息。为加强不同层次特征图之间的信息传递，采用多层次特征融合单元。不同层次特征图对应区域的激活程度存在较大差异，进行多层次特征图融合时，不同层次特征之间的冲突会干扰信息传递，降低特征融合的有效性。针对不同层次特征冲突的问题，采用动态融合的网络结构，该结构对特征图的空间位置分配不同权重，通过学习筛选有效特征和过滤矛盾信息。

9.3 多尺度生成多抗网络

9.3.2 多尺度GAN结构

3. 多层次特征动态融合单元

首先把不同尺度的特征图上采样调整的到相同的尺寸，并在融合时对不同层次的特征图设置空间权重，寻找最优融合策略，具体可表示为

$$F^* = w_1 \cdot F^{1\uparrow} + w_2 \cdot F^{2\uparrow} + w_3 \cdot F^{3\uparrow} + w_4 \cdot F^{4\uparrow}$$

式中， $F^{i\uparrow}$ 代表第i个特征图经过上采样调整到统一尺寸后的标准特征图，所有层次的特征经过自适应权重分配的动态融合输出最后的特征图 F^* 。

权重 w_i 的学习方式，如多层次特征动态融合单元结构图所示。标准特征图的空间信息经过下采样卷积层被压缩，将对应的四个不同层次特征图的压缩卷积层级联起来，使用 1×1 的卷积映射同一位置的特征信息，最后通过Softmax函数标准化网络参数得到空间权重信息，即

$$w_i = \frac{\exp(w'_i)}{\sum \exp(w'_i)}$$

经过学习得到的特征图自适应空间权重 $w_i \in [0,1]$ 且总和为1。

9.3 多尺度生成对抗网络

9.3.2 多尺度GAN结构

4. 损失函数

在训练过程中，损失函数衡量生成图像与真实图像的差别，GAN损失函数包括对抗损失与内容损失。其中对抗损失采用LSGAN中的对抗损失，即

$$Loss_{GAN} = \mathbb{E}[\|\log(D(I_{gt})) - 1\|_2^2] + \mathbb{E}[\|\log(D(I_{gen}))\|_2^2]$$

式中， I_{gt} 代表真实图像， I_{gen} 代表生成图像。在图像重建的内容损失上，选择生成图像和目标图像的均方差损失 $Loss_{MSE}$ 以获得较高的峰值信噪比，同时为了消除伪影，促进图像高频细节的恢复，使得重构图像具有较高的视觉逼真度，引入视觉损失 $Loss_{perv}$ 。感官损失是通过预先训练的VGG19网络实现的，将生成图像和目标图像分别输入到VGG网络中，然后计算经过VGG网络后对应特征图的欧氏距离。

$$Loss_{MSE} = \frac{1}{M \times N} \sum_{i=1}^m \sum_{j=1}^n \|I_{gt,ij} - I_{gen,ij}\|^2 \quad Loss_{perv} = \sum_{i=1}^L \frac{1}{H_i W_i C_i} \|\phi^{(i)}(I_{gt}) - \phi^{(i)}(I_{gen})\|_2^2$$

式中， $\phi^{(i)}$ 代表预训练的VGG19网络。总的损失函数定义为

$$Loss = \alpha Loss_{MSE} + \beta Loss_{perv} + \gamma Loss_{adv}$$

9.4 GAN实例

利用MNIST数据集构建生成对抗网络产生“以假乱真”的手写字体图像。

1.导入PyTorch框架库及torchvision库

```
import torch
```

```
import torch.nn as nn
```

```
from torchvision import datasets
```

```
from torchvision import transforms
```

```
from torchvision.utils import save_image
```

```
from torch.autograd import Variable
```

```
from torch.utils.data import DataLoader
```

9.4 GAN实例

2.构造数据

```
img_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean=(0.5, ), std=(0.5, ))
])
```

#预处理操作

```
data_transform = transforms.Compose(
    [transforms.ToTensor(),
     transforms.Normalize([0.5], [0.5])])
```

#加载数据

```
train_dataset = datasets.MNIST(root='./data', train=True, transform=data_transform, download=True)
test_dataset = datasets.MNIST(root='./data', train=False, transform=data_transform)
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
```


9.4 GAN实例

3.定义GAN神经网络

```
class discriminator(nn.Module):  
    def __init__(self):  
        super(discriminator, self).__init__()  
        self.discriminator = nn.Sequential(  
            nn.Linear(784, 256),  
            nn.ReLU(True),  
            nn.Linear(256, 128),  
            nn.ReLU(True),  
            nn.Linear(128, 1),  
            nn.Sigmoid()  
        )  
    def forward(self, x):  
        x = self.discriminator(x)  
        return x
```

9.4 GAN实例

3.定义GAN神经网络

```
class generator(nn.Module):  
    def __init__(self,input_size):  
        super(generator, self).__init__()  
        self.generator = nn.Sequential(  
            nn.Linear(input_size, 128),  
            nn.ReLU(True),  
            nn.Linear(128, 256),  
            nn.ReLU(True),  
            nn.Linear(256, 784),  
            nn.Tanh()  
        )  
    def forward(self, x):  
        x = self.generator(x)  
        return x
```

9.4 GAN实例

4.训练GAN神经网络

D = discriminator() #实例化判别器

G = generator(Z) #实例化生成器

loss = nn.BCELoss() #定义二元交叉熵损失函数

d_optimizer = torch.optim.Adam(D.parameters(), lr=0.0001) #定义判别器之优化器

g_optimizer = torch.optim.Adam(G.parameters(), lr=0.0001) #定义生成器之优化器

T = 100 #训练迭代次数

9.4 GAN实例

```
for epoch in range(T):
    for i, (im, _) in enumerate(train_loader):
        num_im = im.size(0)
        #训练判别器
        im = im.view(num_im, -1)
        real_im = Variable(im)
        real_label = Variable(torch.ones(num_im))
        fake_label = Variable(torch.zeros(num_im))
        real_pred = D(real_im).squeeze(-1) #预测真图像的类别标记（理想情况为1）
        d_loss_real = loss(real_pred, real_label) #真图像对应损失（预测类别标记，真实类别标记）
        # real_scores = real_pred
        z_vector = Variable(torch.randn(num_im, Z)) #生成噪声向量
        fake_img = G(z_vector) #生成假图像
```

9.4 GAN实例

```
fake_pred = D(fake_img).squeeze(-1) #判别器对假图像的预测类别标记（理想情况为0）
d_loss_fake = loss(fake_pred, fake_label) #假图像对应损失（预测类别标记，真实类别标记）
# fake_scores = fake_pred
d_loss = d_loss_real + d_loss_fake #真假图像损失之和
d_optimizer.zero_grad() #梯度清零
d_loss.backward() #误差反传
d_optimizer.step() #更新参数
```

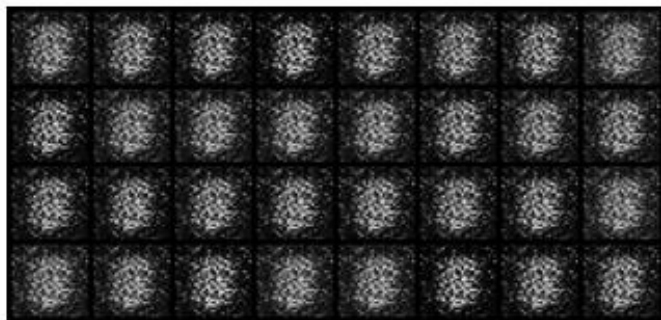
#训练生成器

```
z_vector = Variable(torch.randn(num_im, Z)) #生成噪声向量
fake_im = G(z_vector) #生成假图像
fake_pred = D(fake_im).squeeze(-1) #判别器对假图像的预测类别标记
g_loss = loss(fake_pred, real_label) #计算损失
g_optimizer.zero_grad() #梯度清零
g_loss.backward() #误差反传
g_optimizer.step() #更新参数
```

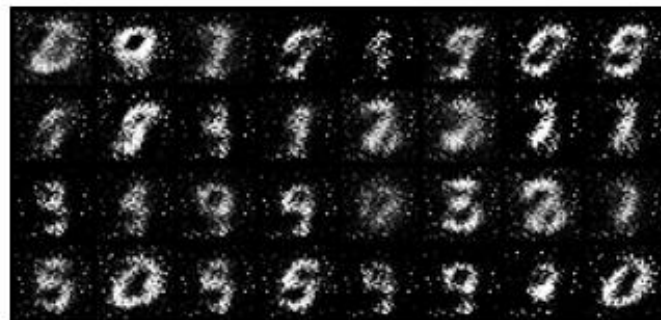
9.4 GAN实例

```
if (i + 1) % 100 == 0:
    print('Epoch [{}/{}], d_loss: {:.6f}, g_loss: {:.6f} '.format(
        epoch, T, d_loss.data.numpy(), g_loss.data.numpy(),))
if epoch == 0:
    real_images = to_im(real_im.data)
    save_image(real_images, './results/real_images.png')
fake_images = to_im(fake_img.data)
save_image(fake_images, './results/fake_images-{}.png'.format(epoch + 1))
```

9.4 GAN实例



(a) 初始噪声生成的图像



(b) 迭代 20 次生成的图像



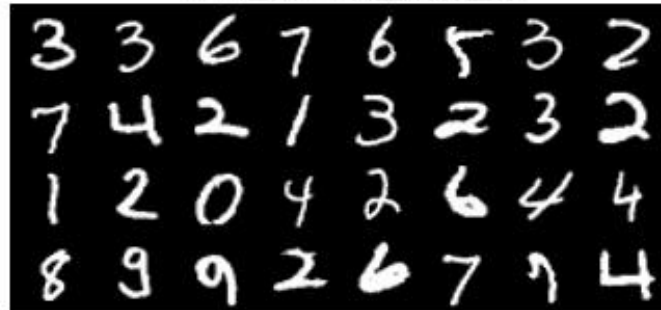
(c) 迭代 50 次生成的图像



(d) 迭代 80 次生成的图像



(e) 迭代 100 次生成的图像



(f) 真实图像

生成对抗网络主要用于在已知数据的基础上生成可靠的或“以假乱真”的新数据，其结构由生成器与判别器两部分构成，而由于其主要解决样本真伪问题，因而，相应的损失函数通常采用二元交叉损失函数。

左图所示生成器旨在生成可以让判别器无法判别的样本，而判别器旨在可靠地判别样本的真伪，两者相互对抗之后，判别器将可以根据任何噪声数据生成“以假乱真”的数据，在实际应用中，生成对抗网络可以根据在已知数据集的基础上对其进行扩展，可在一定程度上解决以数据驱动为特色的深度神经网络所需数据匮乏的问题。

9.4 GAN实例

GAN动物园:

https://gitcode.com/gh_mirrors/th/the-gan-

[zoo?utm_source=csdn_github_accelerator&isLogin=9&from_link=2e49ff47fe0bc30221f2137fcc299edc](https://gitcode.com/gh_mirrors/th/the-gan-zoo?utm_source=csdn_github_accelerator&isLogin=9&from_link=2e49ff47fe0bc30221f2137fcc299edc)